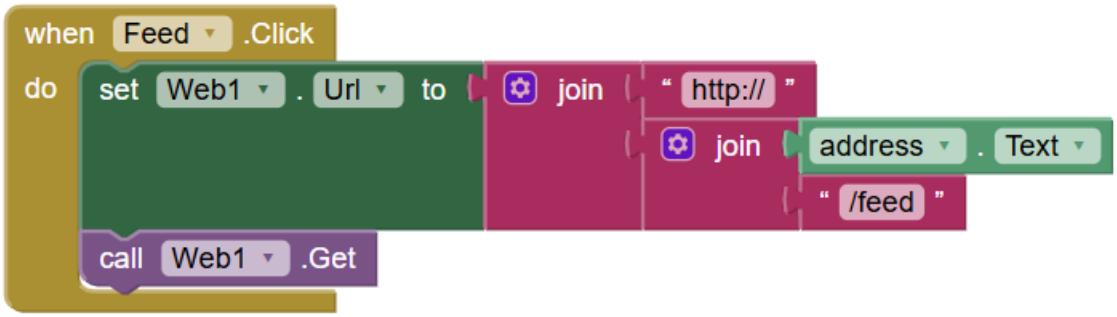
Design Overview

I designed an app-controlled guinea pig feeder. The feeder has three different feed settings (small, medium, and large). I use a Wemos D1 Mini ESP8266 Wi-Fi Board and a 28BYJ-48 Stepper Motor with a ULN2003x Stepper Motor Driver to control the feeder. On the app, the three feed settings/portions are clearly labeled by name and different guinea pigs. When pressed, the motor will step a certain number of times depending on what the input is from the app. The small portion triggers 90 steps, the medium portion triggers 200 steps, and the large portion triggers 400 steps. I used the MIT app inventor to create the app that I used for the feeder.

The app that I created has three different feed options. I will explain how one of them works. The yellow block on the left indicates this is an event handler. This handler triggers once the button “feed” is clicked in the app. This feed corresponds to small portion size. “Feed2” is for the medium and “feed3” is the large portion size. Inside the "do" section is the code that executes when the Feed button is clicked. First, it sets a variable called "Web1.Url" to a constructed URL. The URL is built by joining http:// protocol, the IP address of the feeder, and the /feed which is the endpoint of the server. It then calls "Web1.Get" which sends an HTTP GET request to the URL that was just constructed. All this basically means that when the “feed” button is clicked in the app, it will send a web request to the Wemos D1 Mini ESP8266 Wi-Fi Board at the stored IP address with the "/feed" command. To connect the app to the board, you just type in the IP address of the board, and the app then connects to the board.

(Below is an image of the code block inside the app)



I stuck with almost everything from my original design. However, I was going to add a camera to the feeder that would constantly look at the bowl, so when you wanted to feed your guinea pig, you could see if the bowl was empty or not. I didn’t end up doing this. If I had time, I was going to add this onto the project, but I didn’t have enough time to. Other than that, everything in my design is exactly what I planned to have. I wanted the app to have three different feed options and that is exactly what I did.

This project expands and builds on previous work because I got work with Fusion 360 to 3D print the enclosure and I also got to create an app through the MIT app inventor. I used previous knowledge to write the code and to do the testing for this project. My first week, I ordered the parts and had to wait for them to come in. While waiting for them to come in, I worked on the app. I first had to take time to understand the MIT app inventor website and how to make the app. I then spent this week and some of the 2nd week designing and building the app. My part came in from Amazon on the 3rd week and then I started to do breadboarding and testing. The Wemos D1 Mini ESP8266 Wi-Fi Board required soldering, so I did that and then connected the board to the motor driver. I then used Fusion 360 to help me with my 3D print enclosure and got that printed in week 4 with SERC. Week 5 I combined everything together and got all my stuff to work. Week 6, I did my testing and verification that my guinea pig feeder worked.

References:

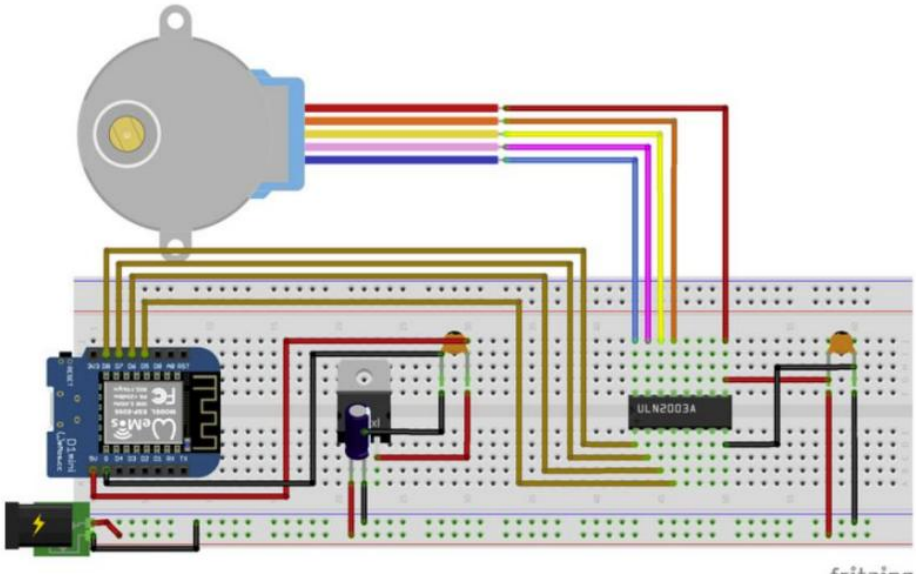
https://www.instructables.com/Pet-Feeder-Controlled-Via-WiFi/

https://projecthub.arduino.cc/Arduino_Genuino/pet-feeder-b81d81#section1

Preliminary Design Verification

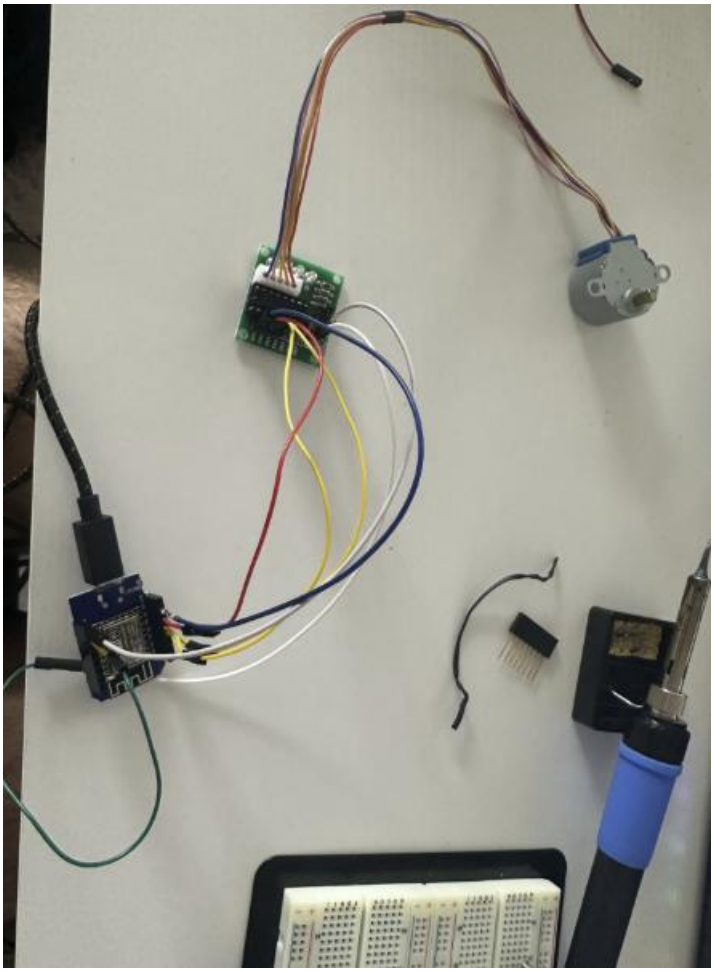
In order to do my preliminary testing, I had to first solder the Wemos D1 Mini ESP8266 Wi-Fi board so that I could connect components to it.

(Below is an image of the schematic I was building)



This schematic didn’t use a stepper motor driver and instead build their own on the breadboard. I didn’t need to do this. I was able to connect the Wemos D1 Mini ESP8266 Wi-Fi board to the stepper motor driver directly, which made testing much simpler.

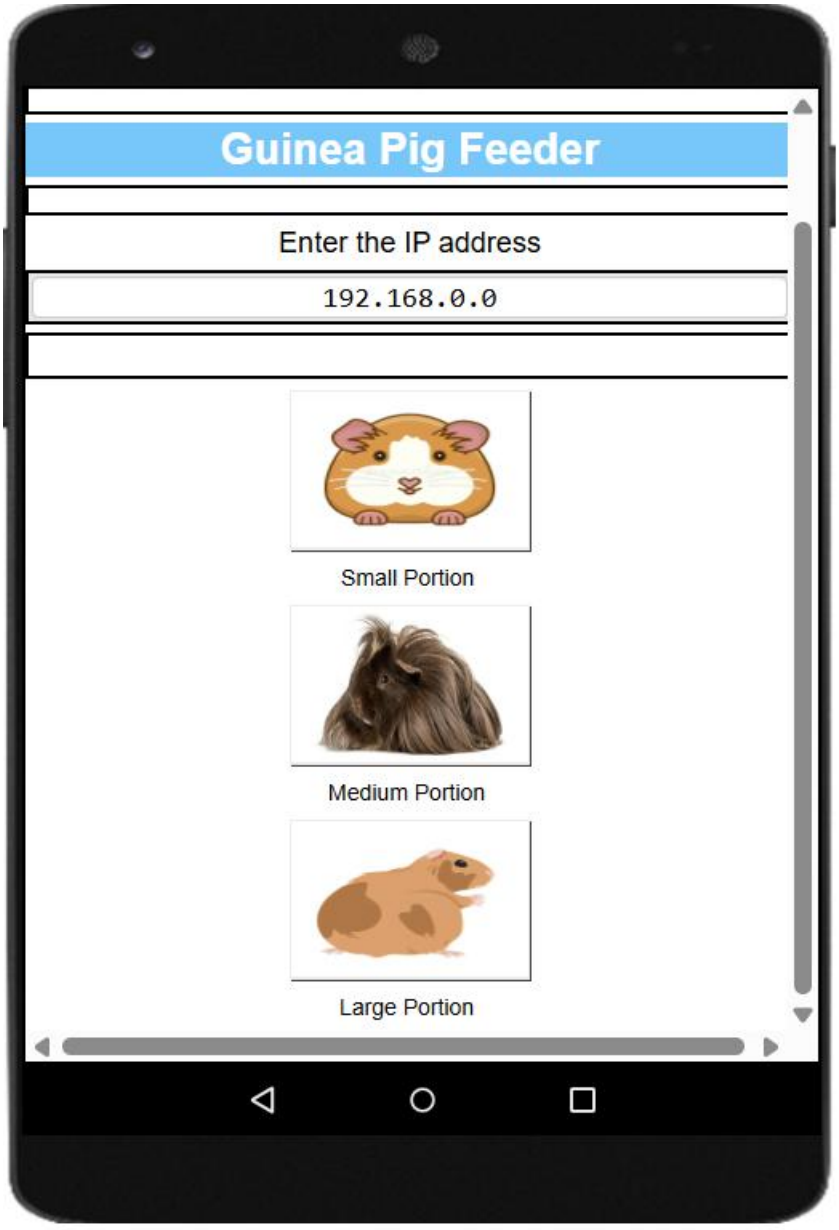
(An image of my Wi-Fi board connected to my stepper motor driver is below)



At this moment, I also thought that I wasn’t going to need to have a PCB board. Something I didn’t account for was the voltage regulator. Once I connected the board to the stepper motor driver, I then downloaded my code onto the board. In my code, I had lines that would print out the IP address of the board so I could connect my app to it. (Below is an image of the code that I used to print the lines to the serial monitor)

```
  }
  Serial.println("");
  Serial.println("WiFi connected");
  Serial.println("Wemos Local IP is : ");
  Serial.print((WiFi.localIP()));
  Serial.print("");
  Serial.println("");
}
```

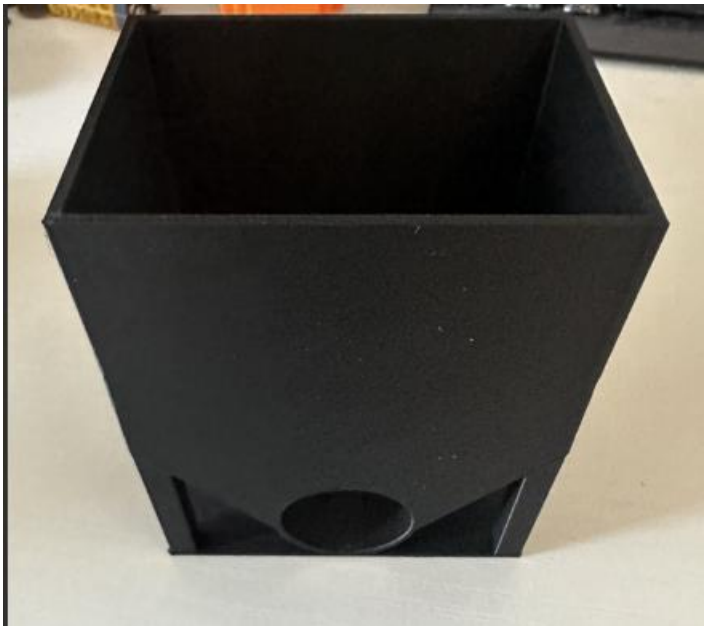
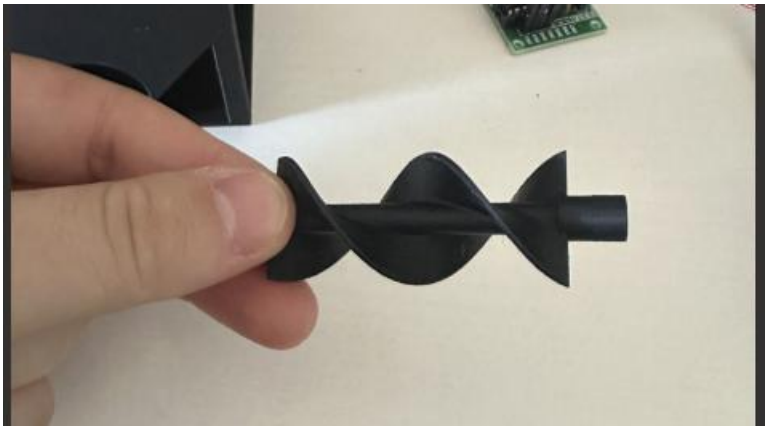
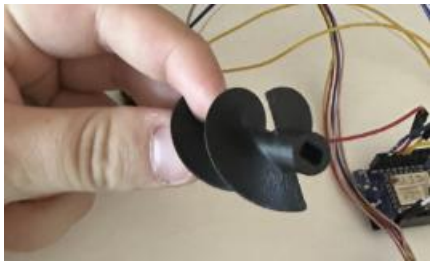
I struggled a lot with this, and it took me a few days to of testing to realize that I couldn't get the IP address of my board while I was connected to "Wireless Pittnet". After a long time of not knowing this, I finally changed to the "ECE DESIGN LAB 2.4" WI-FI and the serial monitor finally displayed my IP address. I then put the IP address into the app and then tested to see if the stepper motor would spin. Each of the three portions worked and spun for different amounts corresponding to the portion sizes. (An image of the app is below)



The top guinea pig is the small portion, the middle guinea pig is the medium portion and the bottom guinea pig is the Large portion.

I then used Fusion 360 for the enclosure. I used a previously 3D enclosure and just scaled it to be proper size for a guinea pig. I then sent the files to SERC and got my enclosure printed. The

(The images below are of the enclosure and the component that connects to the stepper motor)

The two images at the bottom are the part that connects to the stepper motor and this spins to let food out of the feeder when someone presses a feed option on the app.

I finally realized that I needed a voltage regulator, and it was too late to order one as a PCB design, so I made a breadboarded version and connected it up.

Design Implementation

For my final design testing, I combined all my components together. I first put in a 608zz bearing into the spot on the enclosure and then slid the screw through and put in in place. I then put the stepper motor and secured it in the screw. After this, I connected the breadboarded voltage regulator and connected the battery. I tested this in my apartment, where the IP address of my

board was “192.168.1.163”. I typed this IP address into the app and then tested to see if the motor would spin the screw and it did. Each portion spun and dispensed the proper amount of food, verifying that it worked as intended.

The biggest challenge for me in building this was the initial testing and trying to get the IP address of my board while connected to the Wireless Pittet Wi-Fi at school. I didn’t know why I wasn’t able to get my IP address of the board and then finally switched to the ECE DESIGN LAB 2.4 Wi-Fi and it started working. This held me back for a couple of days.

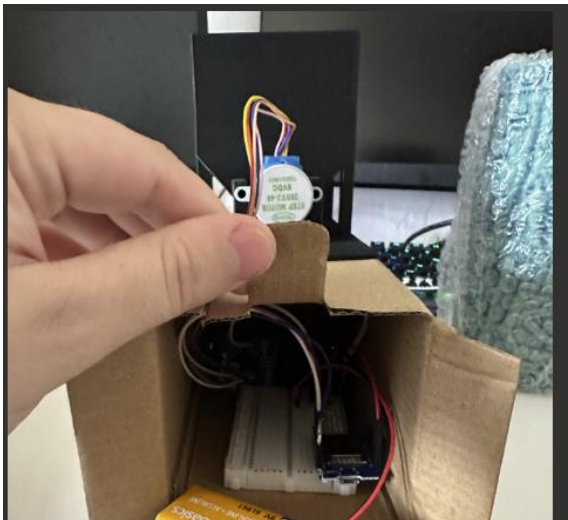
A listing of my components and subcomponents are below:

- Wemos D1 Mini ESP8266 Wi-Fi Board
- 28BYJ-48 Stepper Motor
- ULN2003x Stepper Motor Driver
- 608zz Bearing
- Breadboard
- 3D enclosure
- 3D screw
- MIT app inventor
- iPhone (to control the app)
- 9V battery connected to a voltage regulator

Design Testing

To test my design, I combined all the components together and then loaded my Arduino code onto my board and uploaded it. Inside my serial monitor, it told me the IP address of my board and I then typed that IP address into the app which allowed my app to connect to the board. I then tested each of the three portion sizes to see if it dispensed them properly. There was an issue with the way I was testing. I was using jellybeans to dispense because I didn’t want to buy guinea pig food. I don’t have any at school because my guinea pig isn’t here. The jellybeans kept getting jammed. I solved this issue by changing the code so that the motor would rotate forward for the specified number of steps, pause, and then rotate backward for the same number of steps. This allowed me to not have any more issues with stuff getting jammed. I also switched to mac and cheese noodles because they resemble the size of guinea pig food a little better. After this everything worked with having the small portion only dispensing a small amount of food, the medium portion dispensed a medium portion of food, and the large portion dispensed the greatest amount of food. Since I did test my design very well in the beginning stages, the final testing was very straightforward and simple, just the food part was the only issue. I didn’t have any issues big with doing the final testing and everything worked properly. My design works exactly like I wanted it to by dispensing the food correctly depending on the portion size input.

(Images of the full guinea pig feeder are below)



Video of functioning guinea pig feeder:
https://drive.google.com/file/d/1uHdcLykA6kNM1_ViqSgI_7E3frsRjjIk/view?usp=sharing

Summary, Conclusions and Future Work

This project was cool for me and fun because this is something that I will give to my guinea pig and use to feed him. I never created an app or worked with developing an app. The MIT app inventor was something cool to work with and honestly after I figured out how to work the app inventor, it was cool to design the app and wasn't too difficult in the end. I liked being able to design something that I actually learned a lot from and that I will be able to use regularly. If I did do this again, the one change that I would make to it would be adding the camera to it and maybe using a board that I can access without needing to be connected to the same WI-FI. It would be cool to be away and remotely access the camera and then dispense food. All in all, I really enjoyed this project and this course too. This was 100% my favorite course that I have taken so far at Pitt.

Final Presentation

- Presented in person